

CORE SYSTEM HANDOVER

차트 솔루션 개발

명세서

SDK | 타사 및 설계 명세

3대 핵심 파트 개발 요건



Chart SDK

웹페이지에서 자동으로 인증을 확인하고
화면에 차트를 직접 그려주는 시각화
렌더러



Backend API

권한을 검증하고 DB를 조회하여 차트
맞춤형 데이터로 전송하는 차트 공급자



Admin Console

DB 연결부터 차트 생성, 배포 코드
발급까지 관리하는 제어 센터

Part 1. SDK (sdk.js) 로직 명세

🔍 DOM 스캐닝: [data-chart-id] 마크업 자동 검출

🔑 인증 파싱: JWT or API 토큰

📡 비동기 API 통신: 시각화 설정값 페칭

⚡ Direct Rendering: echarts.init() 직접 바인딩

```
(async () => {  
  // 1. [data-chart-id]를 가진 태그(DOM) 전부 찾기  
  const targets : NodeList<Element> = document.querySelectorAll( selectors: '[data-chart-id]' );  
  
  for (const el : Element of targets) {  
    const chartId : string = el.getAttribute( qualifiedName: 'data-chart-id' );  
    const token : string = el.getAttribute( qualifiedName: 'data-auth-token' );  
  
    // 2. 백엔드 API에 토큰 던져서 데이터 가져오기  
    const res : Response = await fetch( input: `/api/v1/charts/data?chartId=${chartId}`, init: {  
      headers: { 'Authorization': `Bearer ${token}` }  
    });  
    const chartConfig = await res.json(); // ECharts 규칙 데이터  
  
    // 3. IFrame 없이 해당 태그에 차트 바로 그리기  
    const chart = echarts.init(el);  
    chart.setOption(chartConfig);  
  }  
})();
```

```
<div data-chart-id="mc_chart_01" data-auth-token="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... "></div>  
<script src="https://cdn.your-solution.com/sdk.js"></script>
```

Tech Stack: Vanilla JS, Apache ECharts

Part 2. Backend API 서비스 명세

항목 (Category)	상세 명세 (Specification)
Endpoint	GET /api/v1/charts/data (Spring Boot 기반)
Authentication	Secret Key 서명 검증을 활용하는 JWT 인증 체계 or API 토큰 인증 체계
Query Logic	chartId 매핑 데이터베이스 원본 쿼리 실행 및 집계
Data Format	ECharts 전용 JSON 데이터 포맷 반환

Tech Stack: Spring Boot

Part 3. 관리자 페이지 기능 명세

- 📊 **차트 빌더:** SQL 작성기 및 데이터 미리보기
- 📊 **시각화 옵션:** 차트 종류 바꾸고 색상 테마 변경
- 🔗 **코드 제너레이터:** 임베드용 DIV 태그 생성
- 🕒 **데이터 관리:** 데이터 연동 및 관리
- 🔑 **보안 제어:** API 토큰 및 화이트 리스트 관리



DB 배포 및 운용 전략

독립 설치형 (Type A)

전용 PostgreSQL 컨테이너를 함께 패키징합니다.
docker-compose 기반으로 솔루션 구동 전용 로컬 데이터 보관
공간을 형성합니다.

고객 DB 임베디드 (Type B)

자체 인프라 없이, 설치 시 기 운영 중인 고객 DB 주소에
연결합니다. 메타 테이블을 생성하여 임베디드 방식으로
작동합니다.

※ .env 설정파일의 DATABASE_URL 변경만으로 런타임에 DB 계층 자동 변환 처리

DB 테이블 표준 및 접두사

Naming Standards

- 접두사(Prefix): mc_, rpt_, bi_ 등 고유 접두사 필수 사용
- Naming: 소문자 스네이크 케이스 적용 (예: mc_chart_meta)

Conflict Avoidance

- 고객사 운영 테이블 식별 키와의 교집합 원천 배제
- 솔루션 약칭 접두사 사용을 철저히 엄수

Prisma 마이그레이션 자동화

- 🔗 **Prestart Hook 연동:** Next.js 구동 전 prisma migrate deploy 실행 실행 파이프라인
- 🔗 **실시간 DB 구조 진단:** 테이블 부재 시 스키마 자동 컴파일 및 대상 진단
- 🔗 **형상 일치 주의:** 수동 DDL 변경 금지, Prisma 이력을 통한 변경점 추적 관리
- 🛡️ **기존 데이터터 보호:** 독자적인 네임스페이스 영역 격리 엄격 준수

Q&A

Technical Discussion &

Feedback

DEVELOPMENT TEAM | ARCHITECTURE SPEC v1.0